

Task 1: Develop a C program using fork() that creates a parent and child process. Display the process ID(PID), Parent Process ID(PPID), and process states at different stages of execution.

```
ruhi0404@Ruhitha:~/Operating_Systems$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano pra3task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra3task1.c -o pra3task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra3task1
Main process started
PID: 13823

--- PARENT PROCESS ---
Parent PID : 13823
Child PID  : 13824
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID   : 13824
Child PPID  : 13823
Child is running...
Child execution completed
Parent execution completed
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully creates a child process using fork(). The PID and PPID of both the parent and child processes are displayed. The process states are observed at different stages of execution, showing how the parent and child processes are created and executed independently. The experiment demonstrates process creation, parent-child relationships, process identification, and process state changes in Linux.

Task 2: Design an experiment to observe process state transitions(Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top and /proc. Document the observations.

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano pra3task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc pra3task2.c -o pra3task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./pra3task2
Process started
PID: 13852
PPID: 341
Process is running...
Process entering waiting/sleeping state...
Process resumed running...
Process terminating...
ruhi0404@Ruhitha:~/Operating_Systems$ ps -o pid,ppid,state,cmd -p 2600
  PID   PPID  S  CMD
ruhi0404@Ruhitha:~/Operating_Systems$ cat /proc/2600/status
cat: /proc/2600/status: No such file or directory
ruhi0404@Ruhitha:~/Operating_Systems$ top
top - 03:19:45 up 24 min,  1 user,  load average: 0.10, 0.19, 0.34
Tasks:  24 total,   1 running,  23 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,   0.0 ni, 99.9 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem : 15853.6 total, 15336.0 free,   557.8 used,  167.8 buff/cache
MiB Swap:  4096.0 total,  4096.0 free,    0.0 used, 15295.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   22112   12932   9140 S   0.0   0.1   0:14.54 systemd
    2 root        20   0   3180    2208   2072 S   0.0   0.0   0:00.04 init-systemd(Ub
    7 root        20   0   3180    2048   1940 S   0.0   0.0   0:00.00 init
   177 root        20   0   4244    2684   2432 S   0.0   0.0   0:00.03 cron
   178 message+    20   0   9868   5664   4656 S   0.0   0.0   0:03.68 dbus-daemon
   194 root        20   0  17980   8896   7844 S   0.0   0.1   0:01.20 systemd-logind
   219 root        20   0   3124   1972   1836 S   0.0   0.0   0:00.01 agetty
   228 root        20   0 107032  23188  13728 S   0.0   0.1   0:00.79 unattended-upgr
   338 root        20   0   3184   1112    980 S   0.0   0.0   0:00.00 SessionLeader
   339 root        20   0   3200   1252   1108 S   0.0   0.0   0:00.09 Relay(341)
   341 ruhi0404     20   0   6076   5424   3704 S   0.0   0.0   0:00.18 bash
   342 root        20   0   6700   4660   3876 S   0.0   0.0   0:00.03 login
   394 ruhi0404     20   0  20020  10212   8404 S   0.0   0.1   0:00.90 systemd
   395 ruhi0404     20   0  21160   3568   1848 S   0.0   0.0   0:00.00 (sd-pam)
   424 ruhi0404     20   0   6080   5360   3696 S   0.0   0.0   0:00.09 bash
  1524 root        19  -1  50356  15472  14292 S   0.0   0.1   0:08.07 systemd-journal
  1573 systemd+    20   0   91036   7556   6584 S   0.0   0.0   0:00.34 systemd-timesyn
  1626 root        20   0  24888   6096   5004 S   0.0   0.0   0:00.48 systemd-udev
  1680 systemd+    20   0  21468  12712  10460 S   0.0   0.1   0:00.42 systemd-resolve

```

Observation

The experiment successfully demonstrated the different process state transitions in Linux using `ps`, `top`, and the `/proc` filesystem. The process was observed moving between Running (R), Waiting/Sleeping (S), and Terminated (Z or exited) states depending on its execution stage. The `ps` and `top` commands were used to monitor the process state and resource usage, while `/proc/[PID]/status` provided detailed information about the process. The experiment shows how the Linux operating system manages and tracks process states throughout the process lifecycle.